

Assignment No 4

Employee Performance Appraisal System

Table Setup

```
mysql> CREATE TABLE employees (  
->   emp_id INT PRIMARY KEY,  
->   name VARCHAR(50),  
->   salary INT,  
->   rating INT  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> CREATE TABLE appraisal_log (  
->   log_id INT AUTO_INCREMENT PRIMARY KEY,  
->   emp_id INT,  
->   action VARCHAR(20),  
->   old_salary INT,  
->   new_salary INT,  
->   log_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

1) INSERT Trigger (After Adding Employee)

Trigger

```
mysql> CREATE TRIGGER trg_after_insert  
-> AFTER INSERT ON employees  
-> FOR EACH ROW  
-> BEGIN  
->   INSERT INTO appraisal_log(emp_id, action, new_salary)  
->   VALUES (NEW.emp_id, 'INSERT', NEW.salary);  
-> END;  
Query OK, 0 rows affected (0.01 sec)
```

Operation

```
mysql> INSERT INTO employees VALUES (101, 'Amit', 50000, 4);  
Query OK, 1 row affected (0.01 sec)
```

Output — appraisal_log after INSERT

```
mysql> SELECT * FROM appraisal_log;  
+-----+-----+-----+-----+-----+-----+  
| log_id | emp_id | action | old_salary | new_salary | log_date |  
+-----+-----+-----+-----+-----+-----+  
|      1 |    101 | INSERT |      NULL |    50000 | 2024-01-01 10:00:01 |  
+-----+-----+-----+-----+-----+-----+
```

```
1 row in set (0.00 sec)
```

2) UPDATE Trigger (Appraisal Salary Change)

Trigger

```
mysql> CREATE TRIGGER trg_after_update
-> AFTER UPDATE ON employees
-> FOR EACH ROW
-> BEGIN
->   INSERT INTO appraisal_log(emp_id, action, old_salary, new_salary)
->   VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary);
-> END;
Query OK, 0 rows affected (0.01 sec)
```

Operation

```
mysql> UPDATE employees
-> SET salary = 60000
-> WHERE emp_id = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Output — appraisal_log after UPDATE

```
mysql> SELECT * FROM appraisal_log;
+-----+-----+-----+-----+-----+-----+
| log_id | emp_id | action | old_salary | new_salary | log_date |
+-----+-----+-----+-----+-----+-----+
|      1 |    101 | INSERT |      NULL |      50000 | 2024-01-01 10:00:01 |
|      2 |    101 | UPDATE |      50000 |      60000 | 2024-01-01 10:05:22 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3) DELETE Trigger (Employee Removal)

Trigger

```
mysql> CREATE TRIGGER trg_after_delete
-> AFTER DELETE ON employees
-> FOR EACH ROW
-> BEGIN
->   INSERT INTO appraisal_log(emp_id, action, old_salary)
->   VALUES (OLD.emp_id, 'DELETE', OLD.salary);
-> END;
Query OK, 0 rows affected (0.01 sec)
```

Operation

```
mysql> DELETE FROM employees
-> WHERE emp_id = 101;
Query OK, 1 row affected (0.01 sec)
```

Output — appraisal_log after DELETE

```
mysql> SELECT * FROM appraisal_log;
+-----+-----+-----+-----+-----+-----+
| log_id | emp_id | action | old_salary | new_salary | log_date |
+-----+-----+-----+-----+-----+-----+
|      1 |    101 | INSERT |      NULL |    50000 | 2024-01-01 10:00:01 |
|      2 |    101 | UPDATE |    50000 |    60000 | 2024-01-01 10:05:22 |
|      3 |    101 | DELETE |    60000 |      NULL | 2024-01-01 10:08:45 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

4) BONUS: BEFORE UPDATE Trigger (Auto Appraisal Based on Rating)**Trigger**

```
mysql> CREATE TRIGGER trg_before_update
-> BEFORE UPDATE ON employees
-> FOR EACH ROW
-> BEGIN
->   IF NEW.rating = 5 THEN
->     SET NEW.salary = NEW.salary + 10000;
->   ELSEIF NEW.rating = 4 THEN
->     SET NEW.salary = NEW.salary + 5000;
->   END IF;
-> END;
Query OK, 0 rows affected (0.01 sec)
```

Operation

```
mysql> UPDATE employees
-> SET salary = 60000, rating = 5
-> WHERE emp_id = 102;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Output — employees table after BEFORE UPDATE trigger

```
mysql> SELECT * FROM employees WHERE emp_id = 102;
+-----+-----+-----+-----+
| emp_id | name | salary | rating |
+-----+-----+-----+-----+
|    102 | Raj  |  70000 |      5 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
-- Note: Salary automatically becomes 70000 (60000 + 10000)
--       because rating = 5, trigger added 10000 before update.
```

5) BEFORE INSERT Trigger (Validate Salary)**Trigger**

```
mysql> CREATE TRIGGER trg_before_insert
```

```
-> BEFORE INSERT ON employees
-> FOR EACH ROW
-> BEGIN
->   IF NEW.salary < 0 THEN
->     SET NEW.salary = 0;
->   END IF;
-> END;
Query OK, 0 rows affected (0.01 sec)
```

Operation

```
mysql> INSERT INTO employees VALUES (103, 'Sneha', -5000, 3);
Query OK, 1 row affected (0.01 sec)
```

Output — salary corrected to 0 by trigger

```
mysql> SELECT * FROM employees WHERE emp_id = 103;
+-----+-----+-----+-----+
| emp_id | name  | salary | rating |
+-----+-----+-----+-----+
| 103    | Sneha | 0      | 3      |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

6) Viewing All Triggers

```
mysql> SHOW TRIGGERS;
+-----+-----+-----+-----+
| Trigger          | Event | Table    | Timing |
+-----+-----+-----+-----+
| trg_after_insert  | INSERT | employees | AFTER  |
| trg_after_update  | UPDATE | employees | AFTER  |
| trg_after_delete  | DELETE | employees | AFTER  |
| trg_before_update | UPDATE | employees | BEFORE |
| trg_before_insert | INSERT | employees | BEFORE |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

7) Dropping a Trigger

```
mysql> DROP TRIGGER IF EXISTS trg_after_delete;
Query OK, 0 rows affected (0.01 sec)
```

Output — after dropping trg_after_delete

```
mysql> SHOW TRIGGERS;
+-----+-----+-----+-----+
| Trigger          | Event | Table    | Timing |
+-----+-----+-----+-----+
| trg_after_insert  | INSERT | employees | AFTER  |
| trg_after_update  | UPDATE | employees | AFTER  |
| trg_before_update | UPDATE | employees | BEFORE |
| trg_before_insert | INSERT | employees | BEFORE |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```